

System Engineering & AI Optimization: Project Ideation

This document details the architecture, implementation steps, and deployment strategies for two high-impact portfolio projects. These projects are designed to demonstrate a deep understanding of memory management, distributed system bottlenecks, and hardware-level optimization.

Project 1: Write-Through LRU Cache Gateway

In high-throughput environments, repeatedly hitting a primary database or recomputing complex probabilities—such as those found in Bayesian Network inference APIs—can cripple a system. This project demonstrates how to protect backend resources using a custom-built, in-memory cache.

1. Architecture & Tech Stack

- **Core Logic:** Pure Python using `collections.OrderedDict` to maintain $O(1)$ time complexity for reads, inserts, and evictions.
- **Concurrency:** `asyncio` to simulate concurrent user requests without blocking the main thread.
- **Frontend/Deployment:** Streamlit.

2. Streamlit Integration Strategy

Streamlit is excellent for this. We will use `st.session_state` to maintain the LRU Cache across reruns and build an interactive dashboard.

- **Interactive Controls:** Sliders to adjust cache capacity (e.g., 5 to 50 items) and simulated network latency.
- **Visualizer:** A live, dynamic table showing the current state of the cache blocks (what is at the head, what is at the tail).
- **Metrics:** Real-time counters for "Cache Hits," "Cache Misses," and "Average Latency."

3. Step-by-Step Implementation Plan

Phase	Milestone	Technical Details
Phase 1	Core Algorithm	Implement the LRUCache

Phase	Milestone	Technical Details
		class. Override <code>__getitem__</code> and <code>__setitem__</code> . Implement the eviction logic using <code>move_to_end()</code> and <code>popitem(last=False)</code> .
Phase 2	Simulate Backend Delay	Create a dummy "Slow Database" function using <code>time.sleep(2)</code> . Wrap the cache around this function.
Phase 3	Streamlit UI	Build the interface. Use <code>st.columns</code> to put controls on the left and the cache visualization (using <code>st.dataframe</code> or raw HTML blocks) on the right.
Phase 4	Deployment	Push to GitHub. Deploy via Streamlit Community Cloud.

Project 2: Pybind11 8-Bit Quantizer Kernel

When working with massive NLP architectures, such as Transformer models, evaluating inference compute efficiency is paramount. Standard 32-bit floating-point math creates severe memory bandwidth bottlenecks. This project bridges Python and C++ to compress and accelerate matrix operations.

1. Architecture & Tech Stack

- **Backend Math:** Basic C++ using standard vectors to perform 8-bit integer multiplication.
- **The Bridge:** pybind11 to compile the C++ code into a native Python module.
- **Frontend/Deployment:** Streamlit.

2. Streamlit Integration Strategy

Hosting C++ extensions on Streamlit Community Cloud is highly feasible, but it requires instructing the cloud environment to compile your code during deployment.

- **The Build System:** You must include a `setup.py` file that uses Pybind11's build tools, so when Streamlit runs `pip install -r requirements.txt`, it dynamically compiles your C++ kernel into a Linux `.so` file.
- **The Demo UI:** The app will feature a "Run Benchmark" button. It will generate two massive random matrices, run them through standard NumPy (float32), and then through your custom Pybind11 extension (int8).
- **Visuals:** Use `st.bar_chart` to dramatically contrast the Memory Usage (MB) and Execution Time (ms) between the two methods.

3. Step-by-Step Implementation Plan

Phase	Milestone	Technical Details
Phase 1	C++ Kernel	Write a simple C++ function <code>multiply_int8(vector<int8_t> a, vector<int8_t> b)</code> . Keep the memory management basic.
Phase 2	Pybind11 Wrapper	Use the <code>PYBIND11_MODULE</code> macro to expose the C++ function to Python. Ensure you include the correct headers for automatic vector-to-list conversion.
Phase 3	Local Compilation	Write a <code>setup.py</code> file. Run <code>pip install .</code> locally to ensure the C++ code compiles and imports successfully in a standard Python script.
Phase 4	Streamlit UI & Deploy	Build the benchmarking

Phase	Milestone	Technical Details
		dashboard. Add a packages.txt file to your repository containing `cmake` and `gcc` (required for Streamlit's Debian container to compile C++). Deploy.

Summary for Interviews

By hosting these on Streamlit, you bypass the need for recruiters to clone and build your code. You can send them a direct URL where they can immediately see your C++ code outperforming standard Python, and physically watch your LRU cache evicting stale memory blocks.